



universidade
de aveiro

Departamento de Eletrónica, Telecomunicações e Informática
44139 - Computação Móvel
2019-2020

Angelzheimer2



Grupo 3

Sofia Dias
84868

Tiago Rodrigues
84691

Aveiro, 3 de junho de 2020

Resumo

A doença de Alzheimer é uma doença degenerativa cerebral que pode comprometer seriamente as funções cognitivas e a realização de tarefas diárias dos seus portadores. O presente trabalho levou a cabo o desenvolvimento de uma aplicação android direcionada para pessoas que sofrem da doença de Alzheimer, permitindo apoiar o sujeito nas suas tarefas diárias mais básicas através de lembretes, lista de tarefas, localização e ainda permitindo ao respetivo cuidador controlar, gerir e entrar em contacto direto com o paciente.

Índice de Conteúdos

Resumo.....	1
1. Contextualização da Aplicação.....	3
2. A Aplicação AngeLzheimer - Soluções Implementadas	4
2.1. Objetivo	4
2.2. Workflow da Aplicação	4
2.2.1. MainActivity: Inicialização da Aplicação	4
2.2.2. Activity_login: Login ou Registo de utilizadores na Aplicação	4
2.2.3. Main Menu: Menu Principal da Aplicação	7
2.2.4. ActivityFunction: Funcionalidades da Aplicação	9
2.3 Funcionalidades da Aplicação	10
2.3.1. A Fazer	10
2.3.2. Lembretes	12
2.3.3. Conversa.....	13
2.3.4. Localização	13
2.3.5. Informação Pessoal e do Paciente	15
2.3.6. Chamada de Emergência	16
2.4 Adaptações de Layout e Valores	16
2.5. Limitações da Aplicação	18
3. Conclusão	19
4. Recursos da Aplicação	19
5. Referências.....	20

1. Contextualização da Aplicação

A doença de Alzheimer (DA) é uma doença degenerativa cerebral que afeta cerca de 20 em cada mil portugueses, apresentando-se como a forma mais comum de demência no ser humano. Causando degeneração progressiva dos neurónios do cérebro, a DA poderá comprometer seriamente as suas funções cognitivas e a realização de tarefas diárias dos sujeitos.

Um dos sintomas iniciais é a perda de memória, associada muitas vezes à idade do paciente. Esta perda deve-se à formação de tranças neurofibrilares no interior das células e de placas amiloides no espaço extracelular existente entre as células cerebrais.

Com a evolução da doença, o paciente perde a capacidade intelectual, de raciocínio, competências sociais e alterações daquilo que seriam consideradas reações emocionais normais, mantendo apenas alguns comportamentos rotineiros.

Dada a perda de capacidade cognitiva e de memória em sujeitos com DA, a sua qualidade de vida vê-se significativamente reduzida, demonstrando dificuldade na capacidade de localização e orientação bem como memorização de informações pessoais e de contactos de emergência.

2. A Aplicação AngeLzheimer - Soluções Implementadas

2.1. Objetivo

Posta a problemática gerada na presença da DA, propõe-se o desenvolvimento de uma aplicação para smartphone que permita apoiar tanto o paciente como o cuidador nas tarefas diárias do primeiro, nomeadamente nas seguintes funcionalidades:

- Criação e gestão de duas contas com perfiz associados: Paciente e Cuidador;
- Apresentar as informações pessoais, passíveis de serem editadas e guardadas na memória interna do dispositivo móvel e numa base de dados online;
- Mostrar a posição atual e a morada, bem como o percurso mais próximo entre os dois através do sinal de GPS e Google Maps, permitindo ainda ao cuidador saber a última localização do paciente;
- Detetar possíveis quedas através do acelerómetro e informar o cuidador da queda;
- Disponibilizar uma lista de lembretes para apontar detalhes e informações pessoais do paciente;
- Disponibilizar uma lista de tarefas, que poderá ser gerida por ambos o paciente e cuidador e envia uma notificação/alarme aquando da data indicada;
- Disponibilizar um campo de mensagens entre o cuidador e o paciente;
- Disponibilizar uma opção para uma chamada de emergência através de um botão e de forma automática quando detetada uma queda.

2.2. Workflow da Aplicação

2.2.1. MainActivity: Inicialização da Aplicação

A aplicação android inicia-se pela classe *MainActivity*, que, por sua vez, inicia o fragmento *fragment_logo*. Nesse fragmento (Figura 1. a)), é utilizado um método:

```
new Handler().postDelayed(new Runnable() { (... ) }, WaitingTime)
```

onde é averiguado se existe alguma sessão de utilizador iniciada no dispositivo. Caso exista, será averiguado se o respetivo perfil se trata de um Cuidador ou de um Paciente e consequentemente redirecionado para o respetivo Menu Principal do perfil, impossibilitando o utilizador de voltar ao *MainActivity* através do comando:

```
intent.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TASK|Intent.FLAG_ACTIVITY_NEW_TASK).
```

Por outro lado, caso não exista nenhuma sessão iniciada, o utilizador será direcionado para a *Activity_login*.

2.2.2. Activity_login: Login ou Registo de utilizadores na Aplicação

Quando a aplicação é direcionada para a *Activity_login*, recorre-se a um objeto *ViewPager* [1] para adicionar os fragmentos *fragment_login* e *fragment_register*.

No fragmento `fragment_register` (Figura 1. c)), deverá ser o cuidador a efetuar o registo de perfis do próprio e do paciente. Para tal, este deverá indicar o email e password de ambos os utilizadores e proceder ao correspondente registo. Para o efeito, foram utilizadas 4 validações, que informam o utilizador caso uma delas não tenha sido validada e o respetivo erro:

- Campo de email vazio;
- Campo de Password vazio;
- Password inferior a 6 dígitos;
- Email já utilizado.

Após validadas as credenciais, o registo na aplicação de ambos os utilizadores é realizado através da *Firebase Authentication* [2], recorrendo ao método:

```
mAuth.createUserWithEmailAndPassword(email, pass).addOnCompleteListener((Activity) context, new OnCompleteListener<AuthResult>() { (...) }).
```

Após esse processo, guardam-se as informações padrão de cada utilizador num *HashMap*, nomeadamente UID (ID único de cada utilizador obtido a partir do *Firebase Authentication*), Tipo de Utilizador (Cuidador ou Paciente), Nome, Email, Morada, Contacto, Contacto de Emergência e IDHomólogo (ID do Cuidador/Paciente Homólogo correspondente a esse perfil).

Por fim, armazenam-se os respetivos dados online na base de dados na *Firebase Firestore* [3] (Código 1) e internamente recorrendo ao objeto *SharedPreferences* [4].

```
public static void SaveUserData(Map<String, Object> user, String userID) {
    FirebaseFirestore db = FirebaseFirestore.getInstance();

    final DocumentReference documentReference = db.collection("users")
        .document(userID);
    documentReference.set(user).addOnSuccessListener(new
    OnSuccessListener<Void>() {
        @Override
        public void onSuccess(Void aVoid) {
            Log.d("FIRESTORE", "DocumentSnapshot added with ID: " +
            documentReference.getId());
        }
    })
        .addOnFailureListener(new OnFailureListener() {
            @Override
            public void onFailure(@NonNull Exception e) {
                Log.w("FIRESTORE", "Error adding document", e);
            }
        });
}
```

Código 1. Armazenamento de dados do utilizador no *Firebase Firestore*

Por sua vez, no fragmento `fragment_login` (Figura 1. b)), o utilizador deverá colocar o seu email de registo e respetiva password para iniciar sessão na aplicação. Para o efeito, foram utilizadas as 3 primeiras validações anteriormente referidas e ainda uma validação

“Combinação Email + Password errada” e, após validadas as credenciais, o login na aplicação é realizado através da *Firebase Authentication*, recorrendo ao método:

```
mAuth.signInWithEmailAndPassword(email, pass).addOnCompleteListener((Activity) context,  
new OnCompleteListener<AuthResult>() { (...) }).
```

Após esse processo, as informações do utilizador são obtidas do *Firebase Firestore* (Código 2) e guardadas internamente através do objeto *SharedPreferences*.

```
FirebaseFirestore db = FirebaseFirestore.getInstance();  
DocumentReference documentReference=db.collection("users")  
    .document(userID);  
documentReference.get().addOnCompleteListener(new  
OnCompleteListener<DocumentSnapshot>() {  
  
    @Override  
    public void onComplete(@NonNull Task<DocumentSnapshot> task) {  
  
        if (task.isSuccessful()) {  
            DocumentSnapshot document = task.getResult();  
            if (document.exists()) {  
                shared(document); // Guardar internamente com SharedPreferences  
                Log.d("FIRESTORE", "DocumentSnapshot data: " +  
document.getData());  
            } else {  
                Log.d("FIRESTORE", "No such document");  
            }  
        } else {  
            Log.d("FIRESTORE", "get failed with ", task.getException());  
        }  
    }  
}
```

Código 2. Obtenção de dados do utilizador no *Firebase Firestore*

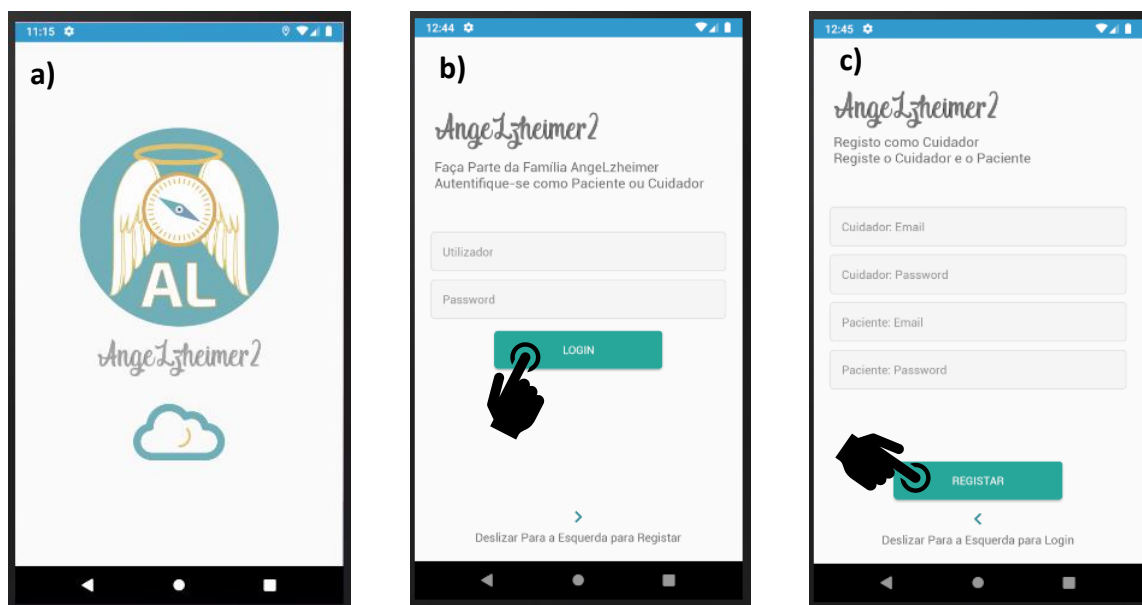


Figura 1. a) Fragmento de apresentação; b) Fragmento de Login; c) Fragmento de Resgisto

Por fim, dependendo do tipo de perfil do utilizador, este será direcionado para o respetivo Menu Principal.

2.2.3. Main Menu: Menu Principal da Aplicação

Dependendo do tipo de perfil, o utilizador será direcionado para o a Atividade de menu principal do Paciente ou do Cuidador (Figura 2), que por sua vez adicionam o fragmento correspondente.

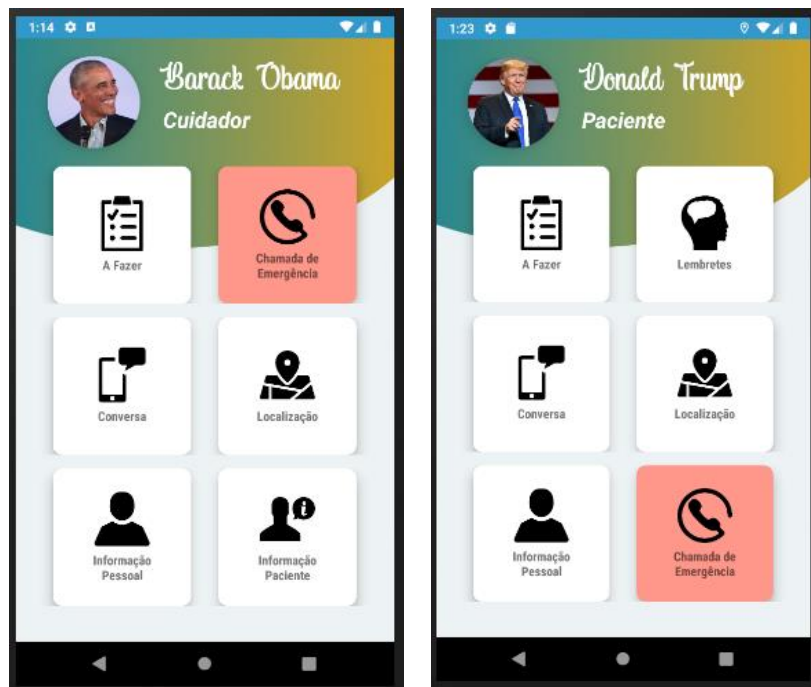


Figura 2. Menu Principal do Cuidador e do Paciente

Em ambos os perfis, o layout está organizado em vários objetos `CardView` contidos em objetos `LinearLayout`, possuindo ainda objetos `TextView` e `ImageView`, que apresentam a informação pessoal do utilizador e a sua fotografia de perfil.

Para trocar a imagem de perfil do utilizador (Figura 3), este poderá fazer um “long click” na sua imagem e, aí, aparecerão as opções para alterar pela câmara ou pela galeria. Após selecionada a foto, esta é transformada num objeto `Bitmap`, carregada para o *Firestore Storage* [5] (Código 3) e atualizada no `ImageView`.

```
final StorageReference reference = FirebaseStorage.getInstance().getReference()
    .child("profileImages")
    .child(UID + ".jpeg");
reference.putBytes(baos.toByteArray())
    .addOnSuccessListener(new OnSuccessListener<UploadTask.TaskSnapshot>() {
        @Override
        public void onSuccess(UploadTask.TaskSnapshot taskSnapshot) {
            Log.w("IMAGE", "Imagem Carregada com sucesso");
            getDownloadURL(reference);
        }
    })
    .addOnFailureListener(new OnFailureListener() {
        @Override
        public void onFailure(@NonNull Exception e) {
            Log.w("IMAGE", "onFailure: ", e);
        }
    });
```

Código 3. Atualização da Imagem de Perfil do utilizador na *Firestore Storage*

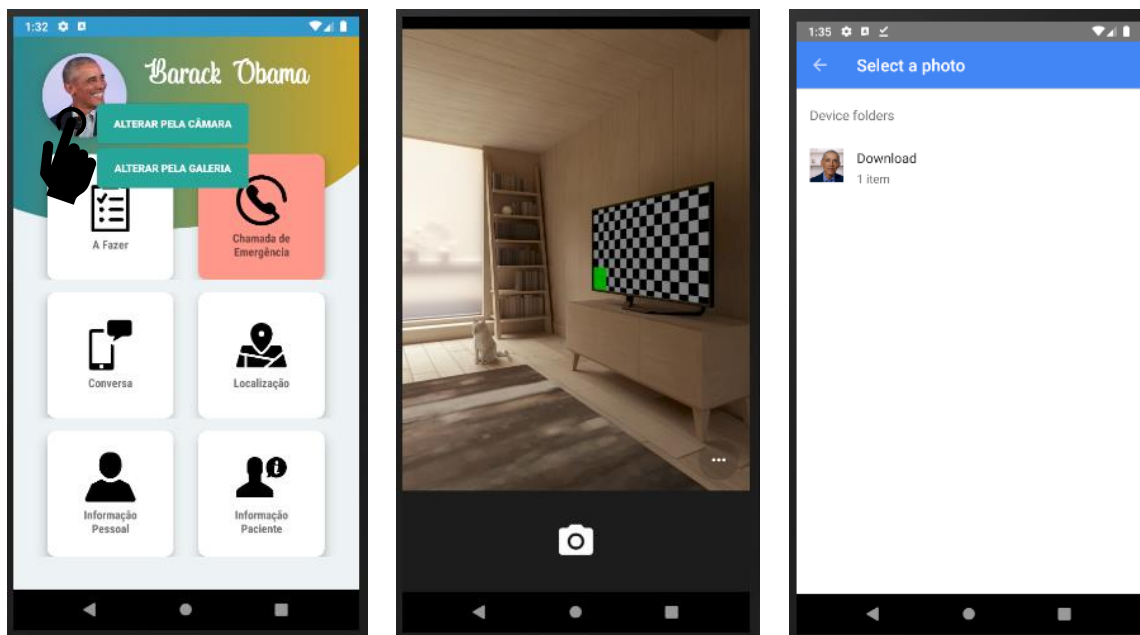


Figura 3. Atualização da Imagem de Perfil do utilizador

No perfil de Paciente, o utilizador tem a possibilidade de seleccionar as opções “A Fazer”, “Lembretes”, “Conversa”, “Localização”, “Informação Pessoal” e “Chamada de Emergência”. Por sua vez, o cuidador tem a possibilidade de seleccionar as opções “A Fazer”, “Chamada de Emergência”, “Conversa”, “Localização”, “Informação Pessoal” e “Informação Paciente”. Independentemente da opção que escolha, o utilizador será sempre direccionado para a atividade *ActivityFunction* com a String da opção escolhida como EXTRA do Intent.

Por fim, caso o utilizador seja um Paciente, é gerada uma nova *Thread* (Código 4), onde é obtida a localização atual do paciente através do sinal GPS, cuja metodologia será explorada adiante, e as respetivas coordenadas guardadas na *Firebase Firestore*.

```
public static boolean isRecursionEnable = true;
void runInBackground() {
    if (!isRecursionEnable)
        return;

    new Thread(new Runnable() {
        @Override
        public void run() → { getLocation(); }).start();
    }
}
```

Código 4. Thread como executável em background

Nesta atividade, é ainda implementado o algoritmo de deteção de queda através dos dados recolhidos pelo acelerómetro (*Sensor.TYPE_ACCELEROMETER*) [6], recorrendo, para isso, aos valores obtidos do *sensorEvent.values* [7], [8].

Este algoritmo (Código 5) calcula o módulo da aceleração para cada conjunto de valores recolhidos e adicionando-o à lista *a*, e utiliza um método de *thresholds* [9] para deteção da queda, determinando que esta ocorreu caso em dois valores consecutivos da lista *a*, *x* e *y*, se verifique que $x < 7.5$ e $y > 15.6$. Verificando-se estas condições, é efetuada uma chamada para o contacto de emergência do utilizador.


```

@RequiresApi(api = Build.VERSION_CODES.N)
@Override
public void onSensorChanged(SensorEvent sensorEvent) {
    Sensor mySensor = sensorEvent.sensor;

    if (mySensor.getType() == Sensor.TYPE_ACCELEROMETER) {
        float x = sensorEvent.values[0];
        float y = sensorEvent.values[1];
        float z = sensorEvent.values[2];
        float ac =(float)sqrt(Math.pow(x, 2) + Math.pow(y, 2) + Math.pow(z, 2));

        long curTime = System.currentTimeMillis();

        if (a != null && a.size() > 5) {
            // Detecção da queda
            Log.d("QUEDA", a.toString());
            for (int i = 0; i < n; i++) {
                if ((float) a.get(i) < lowThreshold) {
                    for (int j = i + 1; j < n; j++) {
                        if ((float)a.get(j) > highThreshold) {
                            fallDetect = true;
                            SharedPreferences sharedPreferences =
this.getSharedPreferences(SHARED_PREFS, MODE_PRIVATE);
                            String numero =
sharedPreferences.getString("ContactoEmergencia", "112");
                            chamar(numero);
                        } else {
                            fallDetect = false;
                        }
                    }
                } else {
                    fallDetect = false;
                }
            }
        }
        a.removeAll(a);
    }
}

```

Código 5. Algoritmo para detecção de queda

2.2.4. ActivityFunction: Funcionalidades da Aplicação

A atividade ActivityFunction recebe a opção selecionada no Menu Principal como EXTRA e, dependendo dessa, adiciona o fragmento da correspondente funcionalidade, excetuando no caso da chamada de emergência, onde executa o *Intent* diretamente, e na opção de lembretes, que inicia uma nova atividade.

2.3 Funcionalidades da Aplicação

A Figura 4 demonstra o comportamento do UI perante a interação do utilizador com a aplicação.

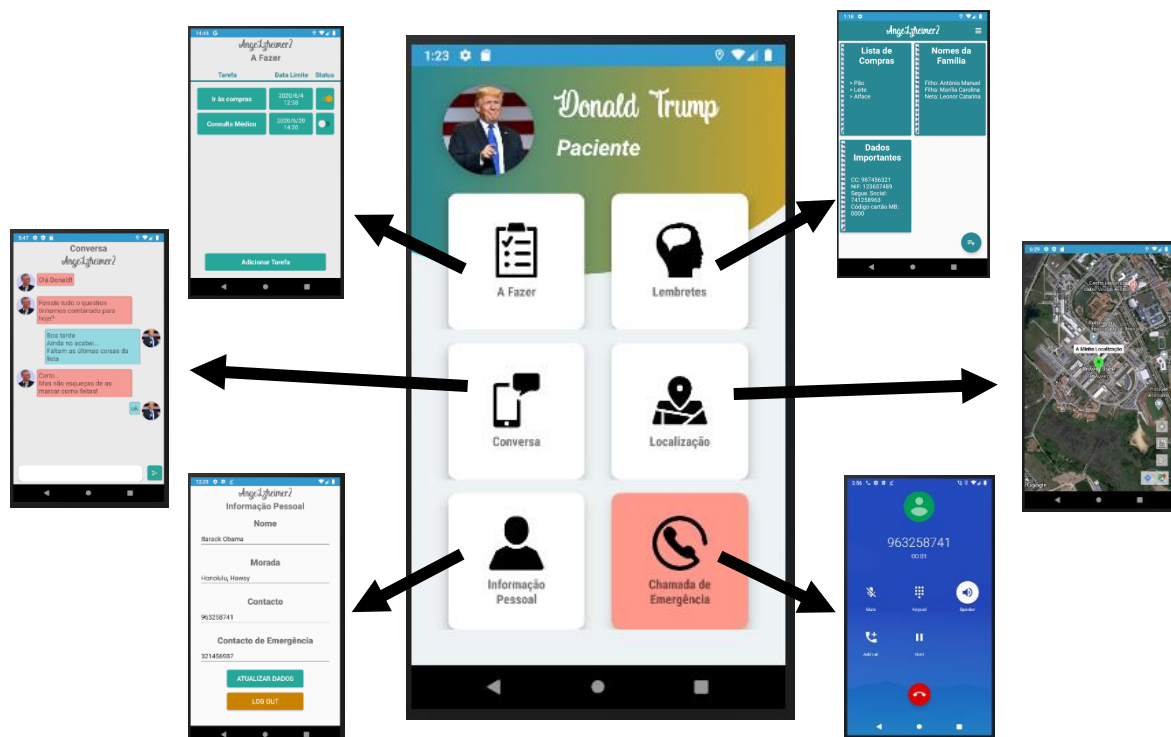


Figura 4. Comportamento do UI da aplicação

2.3.1. A Fazer

A funcionalidade de “A fazer” (*To Do*), demonstrada na consiste numa lista de tarefas planeadas para um determinado momento no futuro. Para este efeito, ambos o Paciente e o Cuidador têm acesso e podem gerir a mesma lista, permitindo-lhes adicionar novas tarefas, editar tarefas anteriores, apagá-las ou, ainda, marcá-las como concluídas.

O layout do fragmento *ToDo* (Figura 5. a)) está organizado em vários objetos *TextView* e um objeto *RecyclerView*, permitindo obter um layout semelhante ao de uma tabela organizada por colunas, e ainda um *Button*, que direciona para uma nova atividade para a criação de uma nova tarefa.

Quando a vista do fragmento é criada, as tarefas já adicionadas são obtidas, e vão sendo atualizadas em tempo real, a partir da *Firestore* e disponibilizadas no *RecyclerView* através de um *adapter*. Quando o utilizador prime o botão “Adicionar Tarefa”, é gerado um novo *Intent* para a atividade que permite adicionar tarefas (Figura 5. b)). Nesta atividade, o utilizador deve indicar o título, a data e a hora da respetiva tarefa, podendo ainda indicar uma descrição, e premir o botão de guardar. Quando tal ocorre, as informações indicadas pelo utilizador são adaptadas para um objeto do tipo *Tasks* (classe personalizada) e carregadas para a *Firestore*, criando ainda um *PendingIntent*, que através de um novo serviço de *BroadcastReceiver*, planeia, caso a tarefa tenha sido planeada para um

momento futuro, uma notificação e um alarme para a data e hora indicadas, tal como demonstrado na Figura 5. d) e e).

Quando a tarefa for concluída, o utilizador tem ainda a possibilidade de a seleccionar como concluída através de um *Switch*, que, quando pressionado, atualiza o estado da respetiva tarefa na *Firebase Firestore*.

Por fim, caso o utilizador deseje editar ou apagar uma das tarefas planeadas, este poderá premir a respetiva tarefa e, logo após, será redirecionado para uma nova atividade de edição (Figura 5. c)), onde pode editar as informações da respetiva tarefa. Quando o botão de guardar é premido, as novas informações são guardadas e a tarefa atualizada na *Firebase Firestore*. Por outro lado, caso o botão de eliminar seja premido, a respetiva tarefa é removida da base de dados online e o utilizador redirecionado novamente para o fragmento principal da funcionalidade.

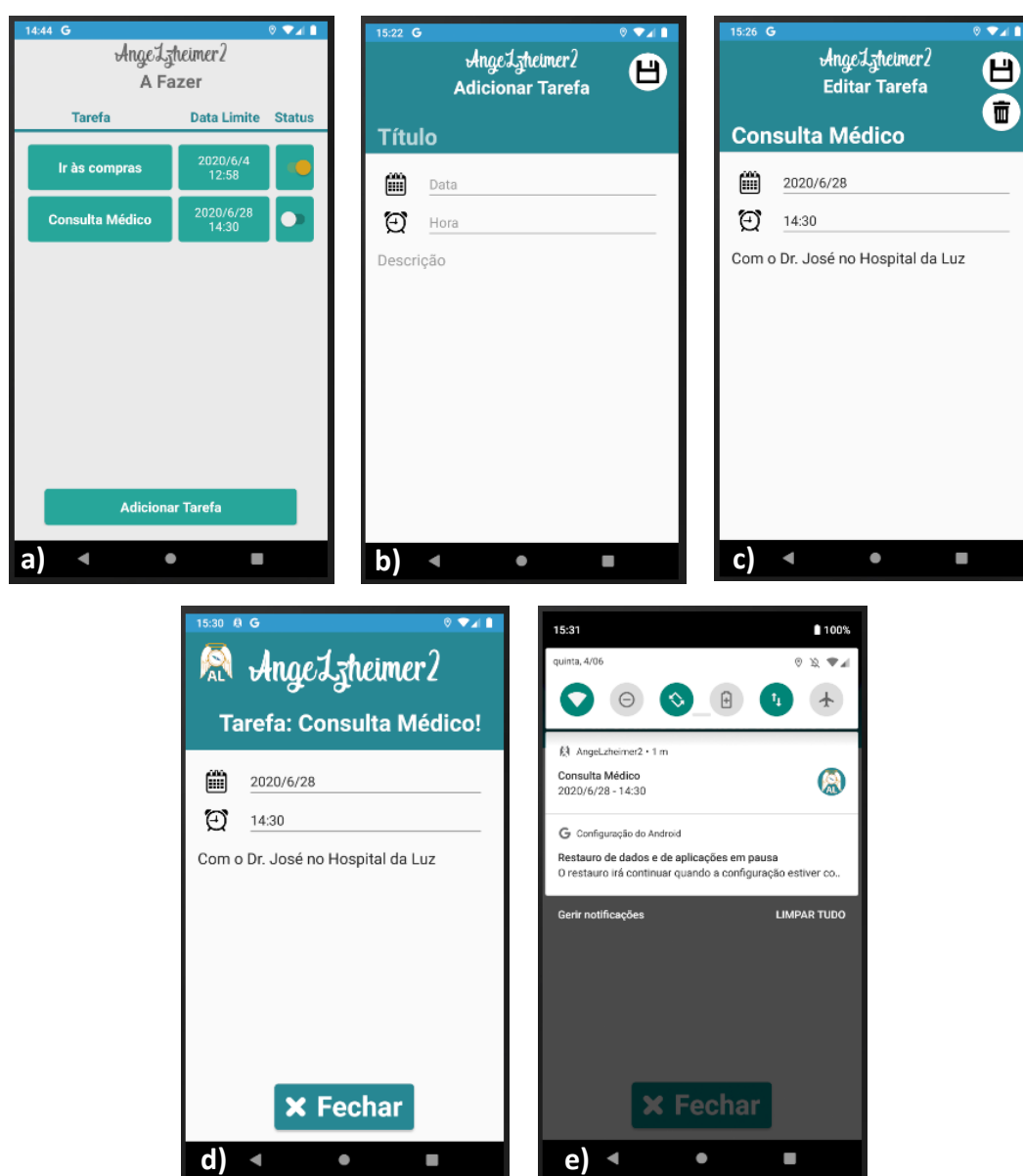


Figura 5. Funcionalidade “A Fazer”: a) Lista de tarefas; b) Adicionar tarefa; c) Editar tarefa; d) Alarme de Tarefa; e) Notificação de Tarefa.

2.3.2. Lembretes

A funcionalidade de lembretes apenas está disponível no perfil do Paciente, foi inspirada no projeto de [10] e consiste num conjunto de blocos de notas onde o utilizador pode apontar, por temas/títulos, informações pessoais, memórias, ou listas de compras. Com isto, o paciente terá acesso a auxiliares de memória que podem ser criados, editados, eliminados e guardados numa base de dados online através da *Firebase Realtime Database* [11].

O layout principal (Figura 6. b) consiste numa *AppBar*, onde é apresentado um ícone para escolher entre vista em lista ou em grelha, num *RecyclerView*, onde são apresentados os vários blocos de notas criados pelo utilizador, e um *Floating Action Button*, que, quando selecionado, direciona o utilizador para uma nova atividade onde este pode criar novos blocos de notas.

Quando o utilizador inicia a atividade (*onStart()*), é apresentado visualmente ao utilizador uma janela de espera *ProgressDialog()* enquanto os blocos de notas são obtidos da *Firebase Realtime Database* (Figura 6. a)). Quando o processo está concluído, recorre-se aos objetos *FirebaseRecyclerOptions* e *FirebaseRecyclerAdapter* [12], que geram um objeto *ViewHolder* para cada bloco de notas e o adiciona ao *RecyclerView* no UI.

Quando o utilizador seleciona a opção de criar um novo bloco de notas ou seleciona um bloco de notas existente é direcionado para uma nova atividade (Figura 6. c)), cujo layout é formado por objetos *EditText*, para edição do título do bloco de notas e a respetiva descrição, e um *Floating Action Button* que permite guardar as alterações na base de dados online *Firebase Realtime Database*.



Figura 6. Funcionalidade de Lembretes: a) Leitura da *Firebase Realtime Database*; b) Vista Principal dos blocos de notas; c) Vista do bloco de notas individual estendido

2.3.3. Conversa

A funcionalidade de conversa (Figura 7) foi inspirada no projeto de [13] e está disponível para ambos o Paciente e Cuidador, permitindo que estes mantenham o contacto através de uma interface de mensagens pela internet.



Figura 7. Funcionalidade de conversa entre Cuidador e Paciente

O layout principal do fragmento consiste num objeto *RecyclerView*, onde são adicionadas as mensagens recebidas e enviadas, um objeto *EditText*, onde o utilizador poderá escrever a mensagem a enviar, e ainda um *ImageButton*, que deverá ser pressionado para enviar a mensagem.

Quando a *View* é criada, o histórico de conversações é obtido da coleção da *Firestore* através de um método semelhante ao do Código 2. Seguidamente, para cada mensagem da conversação, esta é analisada e classificada como “Enviada” ou “Recebida” e, após esta classificação, recorre-se ao objeto *GroupAdapter* [14] para adicionar o respetivo layout de mensagem ao *RecyclerView*.

Quando o utilizador pressiona o *ImageButton* de envio e o texto a enviar não é nulo, recorre-se a um método semelhante ao do Código 1 para adicionar a mensagem como “A Receber” e “A Enviar” na coleção da *Firestore*.

2.3.4. Localização

A opção da localização está disponível para ambos o Paciente e Cuidador, contudo, com funcionalidades relativamente diferentes. Em ambos os casos, recorre-se à API *Google Maps SDK for Android* [15] para mostrar o fragmento do mapa.

No caso do paciente (Figura 8. a) e b)), a opção da localização permite obter e mostrar, no mapa híbrido (*GoogleMap. MAP_TYPE_HYBRID*), a localização atual (a verde), a morada (a vermelho) do paciente e o percurso mais próximo entre as duas posições, permitindo ainda abrir o *Google Maps* para obter direções em tempo real do respetivo percurso.

O layout do fragmento é composto por um fragmento *GoogleMaps* e três *ImageButtons* que permitem, respetivamente, mostrar a localização atual, a posição da morada e as direções entre os dois pontos.

Para tal, recorre-se à API *Google Maps Geocoding* [16] para obter as coordenadas geográficas da morada a partir do endereço indicado pelo utilizador. Por outro lado, para obter a posição atual do utilizador, após a autorização de recurso ao GPS, recorre-se aos objetos *LocationManager* e *LocationListener* [17] e, quando houver uma alteração da posição, o marcador do mapa é substituído pelo mais recente, tal como demonstrado no Código 6.

Por fim, para desenhar o percurso mais próximo entre os dois pontos, recorre-se à API *Google Maps Directions* [18], permitindo obter a informação das direções e pontos da internet como uma *AsyncTask* e, seguidamente, desenhar *Polylines* no mapa formando o respetivo trajeto.

```
@Override
public void onLocationChanged(Location location) {
    if (location != null) {
        double latitude=location.getLatitude();
        double longitude=location.getLongitude();
        Newlocation = new LatLng(latitude, longitude);
        mMap.clear();
        mMap.addMarker(new
MarkerOptions().position(Newlocation).title(getString(R.string.MinhaLocalizacao)
).icon(BitmapDescriptorFactory.defaultMarker(BitmapDescriptorFactory.HUE_GREEN))
);
        mMap.addMarker(new
MarkerOptions().position(MoradaC).title(getString(R.string.Address)).icon(Bitmap
DescriptorFactory.defaultMarker(BitmapDescriptorFactory.HUE_RED)));
        CameraPosition cameraPosition = new
CameraPosition.Builder().target(Newlocation).zoom(16.0f).build();
        CameraUpdate cameraUpdate =
CameraUpdateFactory.newCameraPosition(cameraPosition);
        mMap.moveCamera(cameraUpdate);
    }
}
```

Código 6. Recurso aos objetos *LocationManager* e *LocationListener* para obtenção da localização atual através no método *onLocationChanged*.

Por outro lado, no caso do Cuidador (Figura 8. c)), a opção da localização permite obter a última posição conhecida do Paciente através da *Firestore* e apresentá-la ao mesmo no mapa, facultando ainda a respetiva data e hora.

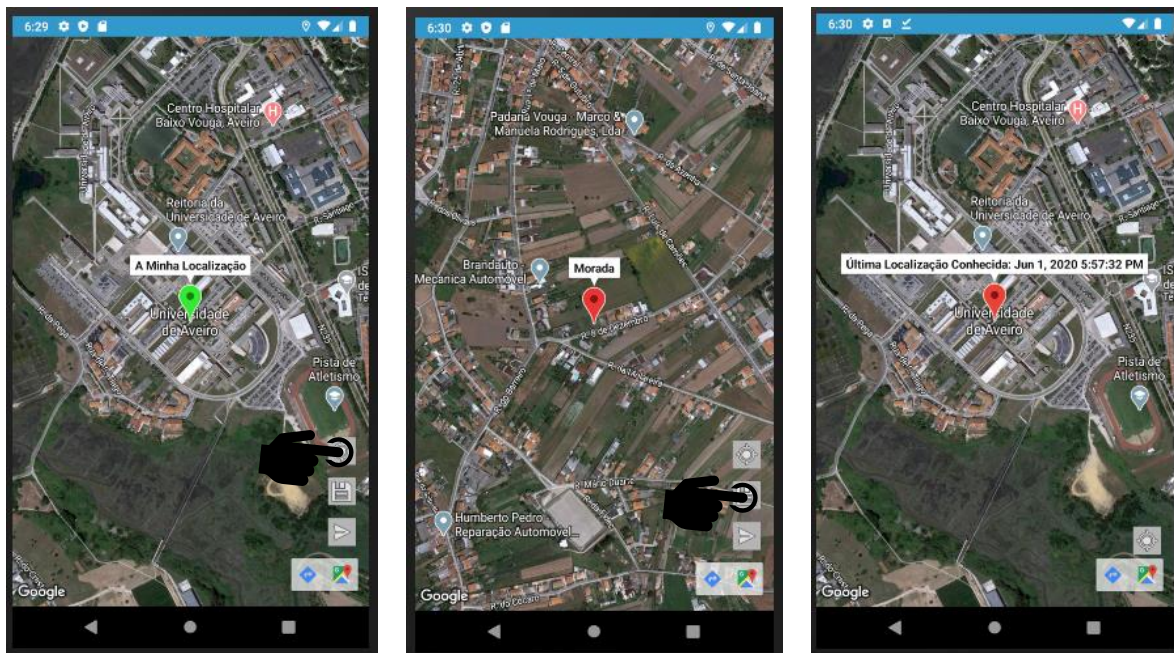


Figura 8. Opção de Localização: a) e b) Funcionalidade do Paciente; c) Funcionalidade do Cuidador

2.3.5. Informação Pessoal e do Paciente

A informação pessoal (Figura 9. a)) é disponibilizada em ambos os tipos de perfil e permite ao utilizador consultar e alterar as suas informações básicas, nomeadamente Nome, Morada, Contacto Pessoal e Contacto de Emergência.

O layout está organizado num objeto *LinearLayout*, onde estão contidos os respetivos objetos *TextView* e *EditText*, e ainda dois objetos *Button*, que permitem atualizar os dados do utilizador e efetuar o Logout da aplicação com o seguinte comando:

```
FirebaseAuth.getInstance().signOut();
```

Quando o UI do fragmento é iniciado (*OnCreateView()*), as informações pessoais anteriormente guardadas no *SharedPreferences* são obtidas e disponibilizadas visualmente ao utilizador. Por sua vez, quando o utilizador seleciona o botão para atualizar as informações, estas, após respetivas validações, são colocadas num objeto *HashMap* e atualizadas internamente com o *SharedPreferences* e online do *Firebase Firestore*.

Por outro lado, o perfil de Cuidador permite ainda a funcionalidade de editar as informações do Paciente homólogo (Figura 9. b)). Para tal, é lido o IDHomólogo do *SharedPreferences* (que estava guardado no perfil do Cuidador), as informações do paciente são lidas da *Firebase Firestore* e apresentadas visualmente ao utilizador. Para atualizar as respetivas alterações, recorre-se a um processo semelhante ao referido anteriormente, com a distinção de se recorrer ao IDHomólogo para guardar as informações online.

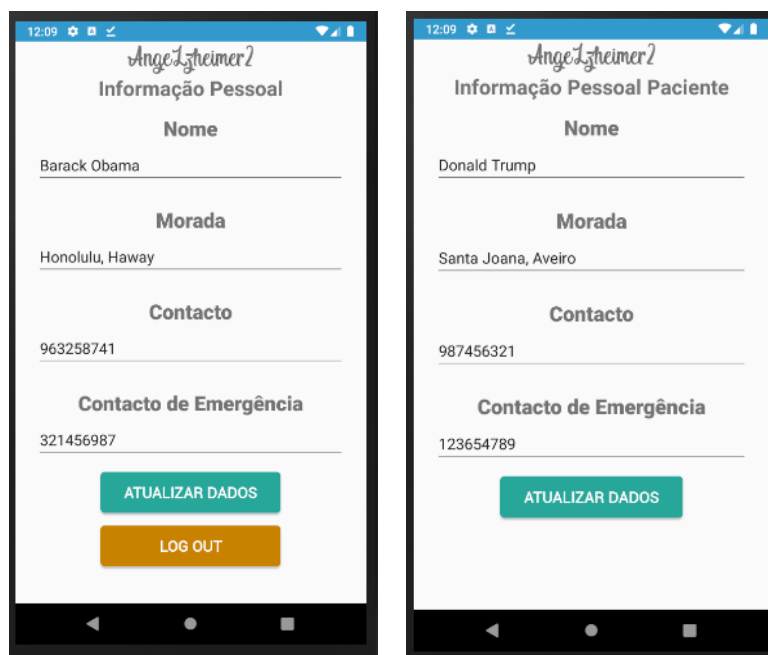


Figura 9. a) Informação Pessoal; b) informação do Paciente

2.3.6. Chamada de Emergência

Caso a chamada de emergência seja selecionada, é efetuada uma chamada para o contacto de emergência (cuidador), obtido a partir do *Firebase Firestore* ou do *SharedPreferences*, e recorrendo ao intent *Intent.ACTION_CALL*.

2.4 Adaptações de Layout e Valores

Tendo em conta a adaptabilidade da aplicação a vários formatos, dimensões de ecrã e orientações, implementaram-se ainda o layout para telemóvel na horizontal (*land*), para tablet na vertical (*sw600dp-port*) e, ainda para tablet na horizontal (*sw600dp-land*), adotando-se um layout *Two pane*, ambos apresentados na Figura 10.

Por sua vez, procedeu-se ainda à adaptação dos valores das *strings* para que a aplicação tivesse suporte de idioma em Português e em Inglês, tal como demonstrado na Figura 11.

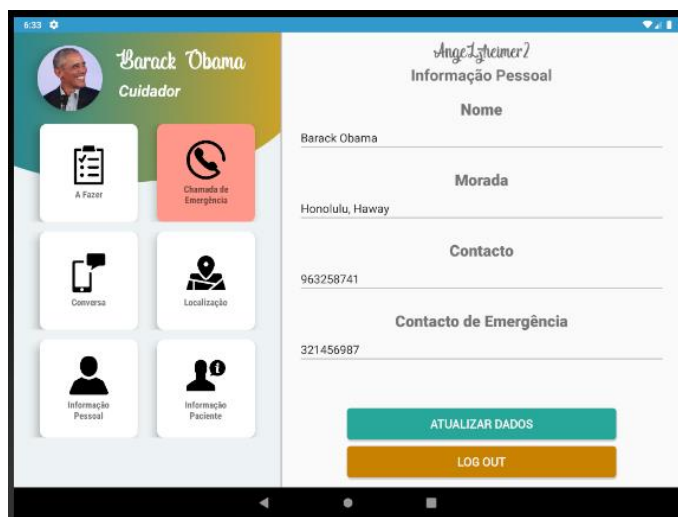
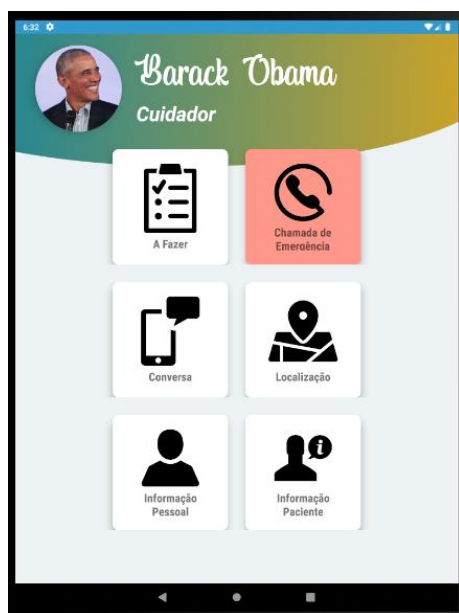
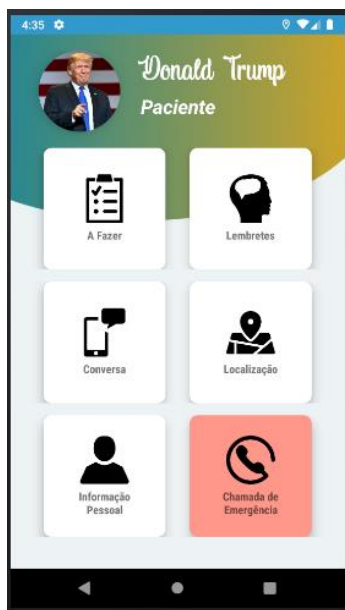


Figura 10. Adaptações de Layout para telemóvel e tablet

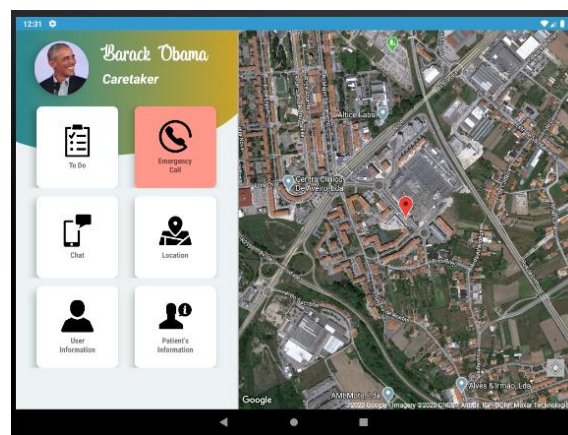
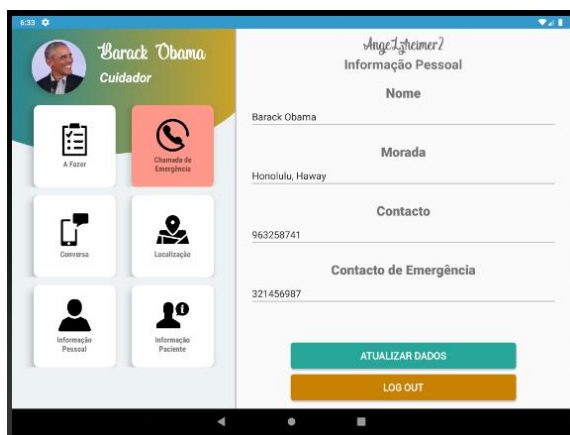


Figura 11. Adaptação de Idiomas: Português e Inglês

2.5. Limitações da Aplicação

Ao longo do desenvolvimento da aplicação foram encontradas diversas dificuldades e limitações, cujas resoluções não puderam ser implementadas por serem trabalhosas e demoradas para o tempo escasso, ou por limitações próprias dos recursos usados.

Em primeiro lugar, na funcionalidade das mensagens/conversa, pretendia-se que o utilizador recebesse, quando não estivesse com o fragmento da conversa aberto, uma notificação alertando para uma mensagem recebida e mostrando o seu conteúdo. Para tal, pensou-se em recorrer ao *Firebase Cloud Messaging*, contudo, tal não foi possível devido à implementação demorosa.

Uma outra limitação da aplicação foca-se na funcionalidade da localização. Pretendia-se que a aplicação fosse capaz de apresentar visualmente o percurso mais rápido entre a localização atual do paciente e a sua morada através da API *Google Maps Directions* e da aplicação de *Polylines*. Posto isto, os métodos foram implementados com base no projeto de [19], mas, uma vez que as cotas da respetiva API são pagas, não é possível obter os pontos do trajeto, aparecendo uma *Toast message* a indicar que não existem pontos a aplicar.

Pretendia-se ainda que todas as funcionalidades fossem aplicadas em fragmentos para que se pudesse adotar um layout “Two Pane” para tablet. No entanto, pretendia-se também aplicar a classe *FirebaseRecyclerAdapter* para a apresentação dos blocos de notas na opção de lembretes. Contudo, quando implementado num fragmento, não aparecia nenhum bloco de notas no *RecyclerView*, pelo que, segundo a informação de [20], se concluiu que esta classe apresenta limitações de aplicabilidade em fragmentos. Como forma de resolução, o método foi implementado numa nova atividade, aparentando funcionar corretamente.

Ainda, a delimitação de uma zona de segurança, recorrendo ao GPS, que permitiria alertar o cuidador caso o paciente a ultrapassasse, poderia ser mais uma medida de segurança a implementar. Além disso, o algoritmo de deteção de queda funcionar mesmo sem a aplicação estar a ser utilizada seria também útil para essa finalidade, bem como a existência de um botão que permitisse ao paciente mostrar algumas informações a alguém caso se encontrasse perdido e pretenda regressar a casa, mesmo que este não reúna as condições cognitivas necessárias a operar facilmente com todas as funcionalidades da aplicação.

Por fim, poderia ser benéfica a utilização de um tutorial que se apresentasse da primeira vez que a aplicação fosse utilizada quer pelo paciente quer pelo cuidador, e que possibilitasse ao segundo requerer que o tutorial (ou parte deste) fosse exibido no perfil do paciente cada vez que este abrisse a aplicação (tendo este a possibilidade de saltar o tutorial se assim o desejasse).

3. Conclusão

Objetivava-se que a aplicação desenvolvida fosse capaz de ser uma ferramenta para melhorar a qualidade de vida de um utilizador com DA, permitindo-lhe organizar as suas tarefas diárias, orientar-se localmente, realizar uma chamada automaticamente em caso de queda e guardar dados pessoais importantes.

Desta forma, fez-se recurso a um conjunto de funcionalidades para o desenvolvimento de uma aplicação *android* simples, organizada, fluída, apelativa e, acima de tudo, capaz de ser usada para os objetivos propostos.

Com isto, tendo um background de programação e de estruturas de dados algo limitado, procuramos aprender a trabalhar e conhecer diversas novas ferramentas para desenvolvimento *Android*, desde APIs, objetos, classes e métodos, tais como os recursos do *Firebase: Authenticator, Firestore, Storage e Realtime Database*, os recursos do *Google Maps* e localização GPS e ainda variados componentes e organização do UI, tais como o *RecyclerView, ViewPager, ViewHolder, FragmentManager*, entre outros.

Tivemos ainda a oportunidade de construir o primeiro contacto com a linguagem de programação JAVA (Android), com novas estruturas de dados e métodos e, ainda, com a gestão de informação em Bases de Dados.

Por fim, aprendemos ainda a trabalhar com métodos de programação e funções assíncronas, bem como gerar e gerir processos em *backend*.

Com o exposto, podemos concluir que a realização deste projeto, embora deveras trabalhoso, foi uma excelente oportunidade para aprender novas técnicas de organização e metodologias de trabalho, que serão certamente úteis no futuro para desenvolvimento de novos algoritmos e software.

Nota: Após a apresentação da Demo da aplicação, foi implementada a funcionalidade “A fazer” na sua totalidade, isto é, a criação e gestão das coleções na *Firebase Firestore*, as atividades, fragmentos e respetivos layouts e ainda a implementação da notificação e alarme.

4. Recursos da Aplicação

Repositório do Projeto:

<https://github.com/tiagoepr/Angelzheimer2.git>

Apk da aplicação:

<https://github.com/tiagoepr/Angelzheimer2/blob/master/angelzheimer2.apk>

Repositório Google Drive (Projeto ZIP, Vídeos e Screenshots):

https://drive.google.com/drive/folders/1BghYkNe_fWcpkQOmuQjHrfGANQyRNwgy?usp=sharing

Chaves de Autenticação:

- Cuidador:
 - Email: teste@teste.com
 - Password: teste123
- Paciente:
 - Email: teste2@teste.com
 - Password: teste123

5. Referências

- [1] “Deslizar entre fragmentos com o ViewPager | Android Developers,” 2020. [Online]. Available: <https://developer.android.com/training/animation/screen-slide>. [Accessed: 02-Jun-2020].
- [2] “Firebase Authentication,” 2019. [Online]. Available: <https://firebase.google.com/docs/auth>. [Accessed: 02-Jun-2020].
- [3] “Cloud Firestore | Firebase,” 2019. [Online]. Available: <https://firebase.google.com/docs/firestore>. [Accessed: 02-Jun-2020].
- [4] “SharedPreferences | Android Developers,” 2019. [Online]. Available: <https://developer.android.com/reference/android/content/SharedPreferences>. [Accessed: 02-Jun-2020].
- [5] “Cloud Storage | Firebase,” 2019. [Online]. Available: <https://firebase.google.com/docs/storage>. [Accessed: 02-Jun-2020].
- [6] “Visão geral dos sensores | Android Developers,” 2019. [Online]. Available: https://developer.android.com/guide/topics/sensors/sensors_overview. [Accessed: 02-Jun-2020].
- [7] “SensorEvent | Android Developers,” 2020. [Online]. Available: <https://developer.android.com/reference/android/hardware/SensorEvent#values>. [Accessed: 02-Jun-2020].
- [8] S. Govender, “Using the Accelerometer on Android,” 2014. [Online]. Available: <https://code.tutsplus.com/tutorials/using-the-accelerometer-on-android--mobile-22125>. [Accessed: 02-Jun-2020].
- [9] T. Tri, H. Truong, and T. Khanh, “Automatic Fall Detection using Smartphone Acceleration Sensor,” *Int. J. Adv. Comput. Sci. Appl.*, vol. 7, no. 12, 2016.
- [10] “My-Diary: Note taking android application aimed to have both a simple interface but keeping smart behavior.” [Online]. Available: <https://github.com/amits999/My-Diary>. [Accessed: 02-Jun-2020].
- [11] “Firebase Realtime Database,” 2019. [Online]. Available:

- <https://firebase.google.com/docs/database>. [Accessed: 02-Jun-2020].
- [12] A. Suda, "How to use FirebaseRecyclerAdpater with latest Firebase Dependencies in Android," 2018. [Online]. Available: <https://medium.com/android-grid/how-to-use-firebase-recycler-adpater-with-latest-firebase-dependencies-in-android-aff7a33adb8b>. [Accessed: 02-Jun-2020].
 - [13] "Tutorial como criar um chat usando firebase." [Online]. Available: <https://github.com/tiago-aguiar/tutorial-chat-firebase>. [Accessed: 02-Jun-2020].
 - [14] "Groupie helps you display and manage complex RecyclerView layouts." [Online]. Available: <https://github.com/lisawray/groupie>. [Accessed: 01-Jun-2020].
 - [15] "Visão geral | Maps SDK for Android | Google Developers," 2020. [Online]. Available: <https://developers.google.com/maps/documentation/android-sdk/intro>. [Accessed: 02-Jun-2020].
 - [16] "Developer Guide | Geocoding API | Google Developers," 2020. [Online]. Available: <https://developers.google.com/maps/documentation/geocoding/intro>. [Accessed: 02-Jun-2020].
 - [17] "LocationListener | Android Developers," 2020. [Online]. Available: <https://developer.android.com/reference/android/location/LocationListener>. [Accessed: 02-Jun-2020].
 - [18] "Get Started | Directions API | Google Developers," 2020. [Online]. Available: <https://developers.google.com/maps/documentation/directions/start>. [Accessed: 02-Jun-2020].
 - [19] G. Mathew, "Drawing driving route directions between two locations using Google Directions in Google Map Android | Knowledge by Experience," 2019. [Online]. Available: <https://www.wingsquare.com/blog/drawing-driving-route-directions-between-two-locations-using-google-directions-in-google-map-android/>. [Accessed: 02-Jun-2020].
 - [20] "FirestoreRecycler adapter is not working on fragment - Stack Overflow," 2017. [Online]. Available: <https://stackoverflow.com/questions/43851451/fragment-firebase-recycler-adapter-is-not-working-on-fragment>. [Accessed: 02-Jun-2020].